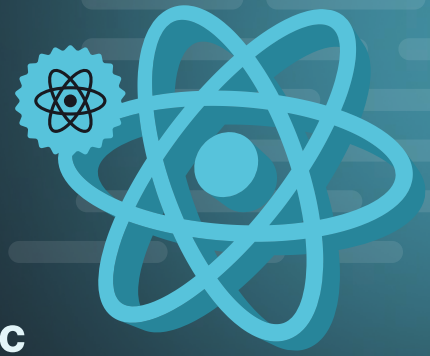


React Concurrent Features Cheatsheet

Modern React patterns for non-blocking updates and better UX.



useDeferredValue

Defer rendering of UI that depends on frequently changing values.

Basic API

```
const deferredValue = useDeferredValue(value);
```

Expensive rendering

Keeps interactions responsive while deferring expensive rendering.

```
const [filter, setFilter] = useState('');
const deferredFilter = useDeferredValue(filter);

<input value={filter} onChange={(e) =>
  setFilter(e.target.value)} />
<ExpensiveList filter={deferredFilter} /> // wrap in
React.memo()
```

Async operations

Prevents jarring content flashes by showing stale results until content is ready.

```
const [query, setQuery] = useState('');
const deferredQuery = useDeferredValue(query);
const isStale = query !== deferredQuery;

<input value={query} onChange={(e) =>
  setQuery(e.target.value)} />
<div style={{ opacity: isStale ? 0.5 : 1 }}>
  <Suspense fallback=<div>Searching...</div>>
    <SearchResults query={deferredQuery} />
  </Suspense>
</div>
```

- Smart prioritization: Defer low-priority updates during frequent changes
- Memo optimization: Works with `React.memo()` wrapped components
- Stale-while-revalidate: Show previous content instead of loading states

useOptimistic

Show optimistic updates immediately while background operations complete.

Basic API

```
const [optimisticState, addOptimistic] =
  useOptimistic(currentState, updateFn);
```

Async user interactions

Shows immediate UI changes during server processing.

```
const [optimisticLiked, setOptimisticLiked] =
  useOptimistic(post.liked);

<button onClick={() => startTransition(async () => {
  setOptimisticLiked(!optimisticLiked);
  await updatePostLike(post.id, !optimisticLiked);
})}>
  {optimisticLiked ? '❤️' : '💔'}
</button>
```

Form submissions

Updates UI instantly while form processes in background.

```
const [optimisticComments, addOptimisticComment] =
  useOptimistic(
    comments,
    (current, newComment) => [...current, {
      ...newComment, isPosting: true
    }]
  );

async function submitAction(formData) {
  addOptimisticComment({ text:
    formData.get('comment'), id: Date.now() });
  await postComment(formData.get('comment'));
}

<form action={submitAction}>
  <textarea name="comment" />
  <button type="submit">Post</button>
</form>
```

- Instant feedback: UI updates immediately, feels native and responsive
- Auto-rollback: Reverts optimistic changes on errors automatically
- Zero boilerplate: No manual pending states or error handling required

Brought to you by



Learn more at certificates.dev/react

Creators of the



React Concurrent Features Cheatsheet

Modern React patterns for non-blocking updates and better UX.

useDeferredValue

Defer rendering of UI that depends on frequently changing values.

Basic API

```
const deferredValue = useDeferredValue(value);
```

Expensive rendering

Keeps interactions responsive while deferring expensive rendering.

```
const [filter, setFilter] = useState('');
const deferredFilter = useDeferredValue(filter);


```

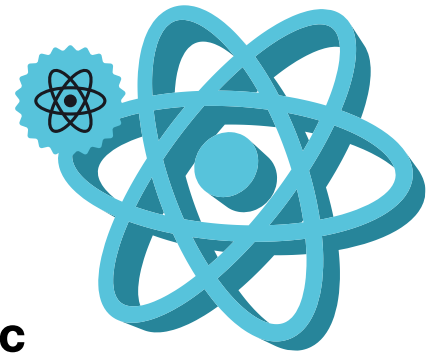
Async operations

Prevents jarring content flashes by showing stale results until content is ready.

```
const [query, setQuery] = useState('');
const deferredQuery = useDeferredValue(query);
const isStale = query !== deferredQuery;


```

- Smart prioritization: Defer low-priority updates during frequent changes
- Memo optimization: Works with `React.memo()` wrapped components
- Stale-while-revalidate: Show previous content instead of loading states



useOptimistic

Show optimistic updates immediately while background operations complete.

Basic API

```
const [optimisticState, addOptimistic] =
  useOptimistic(currentState, updateFn);
```

Async user interactions

Shows immediate UI changes during server processing.

```
const [optimisticLiked, setOptimisticLiked] =
  useOptimistic(post.liked);

<button onClick={() => startTransition(async () => {
  setOptimisticLiked(!optimisticLiked);
  await updatePostLike(post.id, !optimisticLiked);
})}>
  {optimisticLiked ? '❤️' : '💔'}
</button>
```

Form submissions

Updates UI instantly while form processes in background.

```
const [optimisticComments, addOptimisticComment] =
  useOptimistic(
    comments,
    (current, newComment) => [...current, {
      ...newComment, isPosting: true
    }]
  );

async function submitAction(formData) {
  addOptimisticComment({ text:
    formData.get('comment'), id: Date.now() });
  await postComment(formData.get('comment'));
}

<form action={submitAction}>
  <textarea name="comment" />
  <button type="submit">Post</button>
</form>
```

- Instant feedback: UI updates immediately, feels native and responsive
- Auto-rollback: Reverts optimistic changes on errors automatically
- Zero boilerplate: No manual pending states or error handling required

Brought to you by



Learn more at certificates.dev/react

Creators of the

